

Propuesta de Plan de Proyecto Final: Ingeniería en Sistemas de Información

Valentino Russo
LU: 139907
valentinori870@gmail.com

Bahía Blanca, 14 de Agosto de 2025

Bahía Blanca, 14 de Agosto de 2025

Señor Director Decano
Departamento de Ciencias e Ingeniería de la Computación
Dr. Diego C. Martínez
S/D

Me dirijo a Usted y por su intermedio al Consejo Departamental a fin de presentar la Propuesta de Plan de Proyecto Final que se adjunta, conforme a la reglamentación vigente. Adicionalmente adjunto la historia académica para acreditar las condiciones de correlatividad establecidas en el Plan de Estudio 2012 de la Ingeniería en Sistemas de Información.

Sin otro particular lo saluda atte.

Valentino Russo
LU: 139907
email: valentinori870@gmail.com

Propuesta de plan de proyecto final

Ingeniería en Sistemas de Información - Plan 2012

Título/Tema

KnowGraph: Desarrollo de una Aplicación para la Gestión, Consulta y Visualización de Grafos de Conocimiento

Área o Dominio de Aplicación

Ingeniería de Software – Bases de Datos – Web Semántica

Alumno

Russo, Valentino (LU: 139907)

Director

Gomez, Sergio

Introducción

Los grafos de conocimiento se han convertido en una herramienta fundamental para organizar y explotar grandes volúmenes de información interconectada, permitiendo representar entidades y sus relaciones de manera semántica. Sin embargo, las herramientas existentes suelen estar orientadas a usuarios expertos o requieren conocimientos profundos en lenguajes como SPARQL y tecnologías de la Web Semántica.

Este proyecto propone el desarrollo de **KnowGraph**, una aplicación web integral que facilite la gestión, consulta y visualización de grafos de conocimiento. La aplicación permitirá a usuarios con distintos niveles de experiencia cargar datasets en formatos RDF y no RDF (CSV, JSON, Excel, XML), realizar consultas SPARQL mediante un editor asistido, construir visualizaciones interactivas en 2D y 3D (grafos y reducción dimensional), y aplicar razonamiento semántico sobre los datos.

KnowGraph se construirá siguiendo una arquitectura de microservicios dockerizados: un backend en Java con Spring Boot y Apache Jena para el manejo de RDF y consultas SPARQL; un backend en Python con Flask para la importación de datos estructurados y la generación de embeddings mediante técnicas de reducción dimensional (PCA, t-SNE, UMAP); y un frontend en React con TypeScript y Vite, utilizando bibliotecas como ForceGraph, Three.js y D3.js para las visualizaciones. La aplicación se distribuirá como un conjunto de contenedores Docker para facilitar su despliegue y replicabilidad.

Objetivos

Desarrollar una aplicación web funcional y extensible que permita la gestión integral de grafos de conocimiento, desde la carga de datos hasta su consulta, visualización y razonamiento, siguiendo las mejores prácticas de la Web Semántica y la ingeniería de software moderno.

Objetivos específicos

- Implementar un backend basado en Java y Apache Jena que exponga una API REST para la ejecución de consultas SPARQL, la carga/exportación de datasets RDF y la creación de modelos ontológicos y de inferencia.
- Desarrollar un backend en Python con Flask que ofrezca servicios de importación de datos desde formatos estructurados (CSV, Excel, JSON, XML) y los convierta a RDF, así como servicios de reducción dimensional (PCA, t-SNE, UMAP) para visualización de datos multivariados.
- Construir un frontend interactivo en React/TypeScript que integre:
 - Un editor de consultas SPARQL con ejecución y visualización de resultados tabulares.
 - Un asistente visual para la construcción de consultas SPARQL (query wizard).

- Un constructor de triplas para generar consultas de inserción.
 - Visualizaciones de grafos en 2D y 3D basadas en ForceGraph.
 - Visualizaciones de reducción dimensional en 2D y 3D usando D3.js y Three.js.
 - Gestión de datasets (carga local/remota, exportación, limpieza).
- Dockerizar todos los servicios (frontend, backend Java, backend Python) y orquestarlos con Docker Compose para garantizar un despliegue reproducible y portable.
 - Validar la aplicación con datasets de prueba y casos de uso representativos, evaluando su usabilidad, rendimiento y corrección funcional.

Metodología a emplear

El proyecto se desarrollará siguiendo un enfoque iterativo e incremental, con las siguientes fases principales:

1. **Análisis y estudio preliminar:** Revisión de la literatura sobre grafos de conocimiento, RDF, SPARQL, Apache Jena, y tecnologías de visualización. Análisis de herramientas existentes para identificar requisitos funcionales y no funcionales.
2. **Diseño de la arquitectura:** Definición de la arquitectura de microservicios, especificación de APIs REST, diseño de la base de datos RDF (triplestore en memoria), y selección de bibliotecas y frameworks.
3. **Desarrollo del backend Java:** Implementación de los servicios SPARQL, carga de datasets, creación de modelos ontológicos y de inferencia, utilizando Spring Boot y Apache Jena. Pruebas unitarias e integración.
4. **Desarrollo del backend Python:** Implementación de los módulos de importación (CSV, Excel, JSON, XML) y reducción dimensional (PCA, t-SNE, UMAP) con Flask, Pandas, scikit-learn y umap-learn. Pruebas unitarias.
5. **Desarrollo del frontend:** Construcción de la interfaz de usuario con React, TypeScript y Vite. Integración con las APIs de los backends. Desarrollo de los componentes visuales (ForceGraph, D3.js, Three.js). Pruebas de integración.
6. **Integración y pruebas:** Integración de todos los módulos, pruebas de extremo a extremo, verificación de la correcta comunicación entre servicios, y ajustes de usabilidad y rendimiento.
7. **Dockerización y documentación:** Creación de Dockerfiles para cada servicio, configuración de Docker Compose, y redacción de la documentación técnica (manual de usuario, manual de desarrollador).
8. **Redacción del informe final:** Elaboración de la memoria del proyecto, incluyendo introducción, marco teórico, detalles de implementación, resultados y conclusiones.

Actividades y cronograma tentativo

La codificación del programa abarcará aproximadamente 7 semanas y se concentrará en las actividades A3, A4 y A5. El cronograma tentativo para las 16 semanas de duración del proyecto se presenta en la Tabla 1. Cada celda marcada con • indica la semana en que la actividad estará activa.

A1 Análisis de requisitos y estudio de tecnologías (Web Semántica, RDF, SPARQL, Apache Jena, React, etc.)

A2 Diseño de arquitectura y definición de APIs

A3 Implementación del backend Java (Spring Boot + Jena)

A4 Implementación del backend Python (Flask + importación + reducción dimensional)

A5 Implementación del frontend (React + TypeScript + visualizaciones)

A6 Integración de módulos y pruebas de sistema

A7 Dockerización y elaboración de documentación técnica

A8 Redacción del informe final y preparación de la presentación

Cuadro 1: Distribución tentativa de actividades a lo largo de 16 semanas.

Tarea	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
A1	•	•														
A2		•	•													
A3			•	•	•	•										
A4					•	•	•	•								
A5							•	•	•	•						
A6									•	•	•					
A7											•	•	•	•		
A8												•	•	•	•	•

Vinculación con Proyectos de Investigación del DCIC

La propuesta presentada en este proyecto no se encuentra vinculada a ningún proyecto de investigación del DCIC en la actualidad.